

TADs

Algoritmos y estructuras de datos

Elías Cerdeira



¡Bienvenidos
nuevamente!

¿Qué vamos a ver hoy?

- Breve introducción a TADs
- Especificar algunos TADs simples
- **Consultas**



TAD

Pero... ¿qué
era un TAD?



TAD

- Es una representación teórica que nos permite definir un conjunto de **valores**
- Para definir el comportamiento o lo que pueden realizar estos valores usamos **operaciones**
- La parte **abstracta** viene de que nos enfocamos solamente en la **funcionalidad**
- Al igual que la clase pasada, nos dicen **qué** hacen y no **cómo** lo hacen



¿Qué pinta tiene un TAD?

Vamos a definir **qué** hacen los TADs con nuestro lenguaje de especificación

- **Observadores.** Determinan el estado de una instancia del TAD
- **Operaciones.** Modifican o informan sobre el estado de la instancia (**proc**). El efecto que tienen se informa mediante cláusulas **requiere** y **asegura**
- **Predicados y funciones** auxiliares



TAD nombre

obs nombre: tipo

obs nombre: tipo

proc name(params): out_type
 requiere{...}
 asegura{...}

proc name(params): out_type
 requiere{...}
 asegura{...}

pred nombre(params) {
 ...
}

aux name(params): out_type
 ...

Veamos un ejemplo

```
TAD Diccionario<K,V> {  
  obs data: dict<K,V>  
  
  proc diccionarioVacío(): Diccionario<K,V>  
    asegura {res.data = {}}  
  
  proc está(in d: Diccionario<K,V>, in k: K): bool  
    asegura {res = true  $\leftrightarrow$  k  $\in$  d.data}  
  
  proc definir(inout d: Diccionario<K,V>, in k: K, in v: V)  
    requiere {d = D0}  
    asegura {d.data = setKey(D0.data, k, v)}  
  
  proc obtener(in d: Diccionario<K,V>, in k: K): V  
    requiere {k  $\in$  d.data}  
    asegura {res = d.data[k]}  
  
  proc borrar(inout d: Diccionario<K,V>, in k: K)  
    requiere {d = D0  $\wedge$  k  $\in$  d.data}  
    asegura {d.data = delKey(D0.data, k)}  
  
  proc definirRápido(inout d: Diccionario<K,V>, in k: K, in v: V)  
    requiere {d = D0  $\wedge$  k  $\notin$  d.data}  
    asegura {d.data = setKey(D0.data, k, v)}  
  
  proc tamaño(in d: Diccionario<K,V>):  $\mathbb{Z}$   
    asegura {res = |d.data|}  
}
```

En este caso **K** y **V** son tipos genéricos que se instanciarán según el caso. Por ej:
Diccionario<Int,String>



¡Vamos a ver el apunte!

¡Llego primera!
¡Te gano!



¡A resolver ejercicios!



Ahora se pone
divertido...

1er TAD

Ejercicio 1. Especificar en forma completa el TAD `NumeroRacional` que incluya al menos las operaciones aritméticas básicas (suma, resta, división, multiplicación)



- 1) Piensen los observadores
- 2) Especificar las operaciones
- 3) Piensen ahora qué otros observadores podrían usar

Respuesta 1

```
TAD NumeroRacional {
  obs numerador :  $\mathbb{Z}$ 
  obs denominador :  $\mathbb{Z}$ 

  proc nuevoNumeroRacional(in  $n, d : \mathbb{Z}$ ) : NumeroRacional {
    requiere {  $d \neq 0$  }
    asegura {  $res.numerador = n \wedge res.denominador = d$  }
  }

  proc igualdad(in  $n_1, n_2 : NumeroRacional$ ) : bool {
    asegura {  $res = true \leftrightarrow n_1.numerador \times n_2.denominador = n_2.numerador \times n_1.denominador$  }
  }

  proc suma(in  $n1, n2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n1.numerador \times n2.denominador + n2.numerador \times n1.denominador \wedge$ 
       $res.denominador = n1.denominador \times n2.denominador$ 
    }
  }

  proc resta(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n_1.numerador \times n_2.denominador - n_2.numerador \times n_1.denominador \wedge$ 
       $res.denominador = n_1.denominador \times n_2.denominador$ 
    }
  }

  proc multiplicacion(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n_1.numerador \times n_2.numerador \wedge$ 
       $res.denominador = n_1.denominador \times n_2.denominador$ 
    }
  }
}
```

Más TADs

c) Especifique el TAD `DobleCola<T>`, en el que los elementos pueden insertarse al principio o al final y se eliminan por el medio. Debe contener las operaciones

- `nuevaDobleCola` que crea una cola vacía
- `estáVacía` que devuelve true si la cola no contiene elementos
- `encolarAtrás` que agrega un elemento al final de la cola
- `encolarAdelante` que agrega un elemento al principio de la cola
- `desencolar` que elimina el primer elemento de la cola y lo devuelve

1) Piensen los observadores

2) Especificar las operaciones que les piden



Respuesta 3.c

```
c) TAD DobleCola(T) {  
  obs elems : seq(T)  
  proc nuevaDobleCola() : DobleCola(T) {  
    requiere { Verdadero }  
    asegura { |res.elems| = 0 }  
  }  
  
  proc igualdad(in c1, c2 : DobleCola(T)) : bool {  
    asegura { res ↔ c1.elems = c2.elems }  
  }  
  
  proc estaVacía(in c : DobleCola(T)) : bool {
```

```
    requiere { Verdadero }  
    asegura { res = True ↔ |c.elems| = 0 }  
  }  
  
  proc encolarAtrás(inout c : DobleCola(T), in elem : T) {  
    requiere { c = C0 }  
    asegura { c.elems = C0.elems ++ ⟨e⟩ }  
  }  
  
  proc encolarAdelante(inout c : DobleCola(T), in elem : T) {  
    requiere { p = P0 }  
    asegura { agregadoAdelante(C0, c, e) }  
  }  
  
  – Tomo ÷ como división entera  
  proc desencolar(inout c : DobleCola(T)) : T {  
    requiere { c = C0 ∧ |c.elems| > 0 }  
    asegura { res = C0.elems[|C0.elems| ÷ 2] ∧ sacoElDelMedio(C0, c) }  
  }  
  
  pred agregadoAdelante(C0, c : DobleCola(T), e : T) {  
    c.elems[0] = e ∧ |c.elems| = |C0.elems| + 1 ∧  
    (∀ i : ℤ) (0 ≤ i < |c.elems| →L c.elems[i] = C0.elems[i - 1])  
  }  
  
  pred sacoElDelMedio(C0, c : DobleCola(T)) {  
    |c.elems| = |C0.elems| - 1 ∧  
    (∀ i : ℤ) (0 ≤ i < |c.elems| ÷ 2 →L c.elems[i] = C0.elems[i]) ∧  
    (∀ i : ℤ) (|c.elems| ÷ 2 ≤ i < |c.elems| →L c.elems[i] = C0.elems[i + 1])  
  }  
}
```

¡Intervalo!

¡Me hago pipí!



¿Un último TAD?



Ejercicio 9. Especificar TADs para las siguientes estructuras:

a) **Multiconjunto** $\langle T \rangle$

También conocido como **multiset** o **bag**. Es igual a un conjunto pero con duplicados: cada elemento puede agregarse múltiples veces. Tiene las mismas operaciones que el TAD **Conjunto**, más una operación que indica la multiplicidad de un elemento (la cantidad de veces que ese elemento se encuentra en la estructura). Nótese que si un elemento es eliminado del multiconjunto, se reduce en 1 la multiplicidad.

```
c := new MultiConjunto<int>()

agregar(c, 1)
agregar(c, 1)
pertenece(c, 1)  // devuelve true
multiplicidad(c, 1) // devuelve 2

sacar(c, 1)
pertenece(c, 1)  // devuelve true
multiplicidad(c, 1) // devuelve 1
```

1) Pensemos más de un conjunto de observadores

2) ¿Dónde está la diferencia entre TADs en este caso?



Respuesta 9

a) TAD MultiConjunto $\langle T \rangle$ {

```
obs elemsConMult : dicc $\langle T, \mathbb{Z} \rangle$ 

proc conjuntoVacio() : MultiConjunto $\langle T \rangle$  {
  requiere { Verdadero }
  asegura {  $|res.elemsConMult| = 0$  }
}

proc igualdad(in  $c_1, c_2$  : MultiConjunto $\langle T \rangle$ ) : bool {
  requiere { Verdadero }
  asegura {  $res = true \leftrightarrow c_1.elemsConMult = c_2.elemsConMult$  }
}

proc agregar(inout  $c$  : MultiConjunto $\langle T \rangle$ , in  $e$  :  $T$ ) {
  requiere {  $c = C_0$  }
  asegura {
     $(e \in claves(C_0.elemsConMult) \wedge$   

 $c.elemsConMult = setKey(C_0.elemsConMult, e, C_0.elemsConMult[e] + 1))$   

 $\vee$   

 $(e \notin claves(C_0.elemsConMult) \wedge c.elemsConMult = setKey(C_0.elemsConMult, e, 1))$ 
  }
}

proc pertenece(in  $c$  : MultiConjunto $\langle T \rangle$ , in  $e$  :  $T$ ) : bool {
  requiere { Verdadero }
  asegura {  $res = true \leftrightarrow e \in c.elemsConMult$  }
}
```

```
proc eliminar(inout  $c$  : MultiConjunto $\langle T \rangle$ , in  $e$  :  $T$ ) {
  requiere {  $c = C_0$  }
  asegura {
     $e \notin C_0.elemsConMult \wedge c.elemsConMult = C_0.elemsConMult$   

 $\vee$   

 $e \in C_0.elemsConMult \wedge_L ($   

 $C_0.elemsConMult[e] > 1 \wedge$   

 $c.elemsConMult = setKey(C_0.elemsConMult, e, C_0.elemsConMult[e] - 1)$   

 $\vee$   

 $C_0.elemsConMult[e] = 1 \wedge$   

 $c.elemsConMult = delKey(C_0.elemsConMult, e)$ 
  )
  }
}

proc multiplicidad(in  $c$  : MultiConjunto $\langle T \rangle$ , in  $e$  :  $T$ ) :  $\mathbb{Z}$  {
  requiere { Verdadero }
  asegura {
     $e \in claves(c.elemsConMult) \wedge res = c.elemsConMult[e]$   

 $\vee$   

 $e \notin claves(c.elemsConMult) \wedge res = 0$ 
  }
}
```

No, había uno más

Ejercicio 13. Queremos modelar el funcionamiento de un vivero. El vivero arranca su actividad sin ninguna planta y con un monto inicial de dinero.

Las plantas las compramos en un mayorista que nos vende la cantidad que deseemos pero solamente de a una especie por vez. Como vivimos en un país con inflación, cada vez que vamos a comprar tenemos un precio diferente para la misma planta. Para poder comprar plantas tenemos que tener suficiente dinero disponible, ya que el mayorista no acepta fiarnos.

A cada planta le ponemos un precio de venta por unidad. Ese precio tenemos que poder cambiarlo todas las veces que necesitemos. Para simplificar el problema, asumimos que las plantas las vendemos de a un ejemplar (cada venta involucra un solo ejemplar de una única especie). Por supuesto que para poder hacer una venta tenemos que tener *stock* de esa planta y tenemos que haberle fijado un precio previamente. Además, se quiere saber en todo momento cuál es el balance de caja, es decir, el dinero que tiene disponible el vivero.

Tené piedad...



Respuesta 13

```
proc abrirVivero(in montoInicial : Dinero) : Vivero {
  requiere { montoInicial > 0 }
  asegura { res.balance = montoInicial ∧ res.stock = ⟨⟩ }
}

proc igualdad(in V1 : Vivero, in V2 : Vivero) : bool {
  asegura { res = true ↔ V1.balance = V2.balance ∧ V1.stock = V2.stock }
}

proc compraMayorista(inout v : Vivero, in p : Planta, in cantidad : ℤ, in precio : Dinero) {
  requiere { v = V0 ∧ cantidad > 0 ∧ precio > 0 ∧ v.balance ≥ cantidad × precio }
  asegura {
    v.balance = V0.balance - cantidad × precio ∧ (
      seAgregoMercaderiaNueva(V0, v, p, cantidad, precio) ∨
      seAgregoStock(V0, v, p, cantidad, precio)
    )
  }
}
```

```
proc ponerPrecio(inout v : Vivero, in p : Planta, in precio : Dinero) {
  requiere { v = V0 ∧ p ∈ v.stock ∧ precio > 0 }
  asegura {
    v.balance = V0.balance ∧
    v.stock = setKey(V0.stock, p, ⟨V0.stock[p].cantidad, precio⟩)
  }
}

proc vender(inout v : Vivero, in p : Planta) {
  requiere { v = V0 ∧ p ∈ v.stock ∧ v.stock[p] > 0 ∧ v.stock[p].cantidad > 0 }
  asegura {
    v.balance = V0.balance + V0.stock[p].precio ∧
    v.stock = setKey(V0.stock, p, ⟨V0.stock[p].cantidad - 1, V0.stock[p].precio⟩)
  }
}

proc balance(in v : Vivero) : Dinero {
  requiere { Verdadero }
  asegura { res = v.balance }
}
```

¿Qué sigue?

- Con la clase de hoy pueden empezar la **guía 3** (hasta ejercicio 9)
- La clase que viene tenemos **más ejercicios** de TADs de la mano de Román



¡Terminamos!

¡Hagan consultas!

Gracias por
acompañarnos

